

## 20: Dimensionality Reduction

*Date: April 7, 2026*

We're now going to turn our attention to **dimensionality reduction**, an unsupervised learning problem. Here, one is given some data  $\mathbf{x}_1, \dots, \mathbf{x}_n$  (e.g., feature vectors only). While the previous lectures focused on unsupervised clustering (grouping similar data points), dimensionality reduction aims to find a more compact representation of the data, often preserving its essential structure. Sometimes, techniques like PCA are used as a preprocessing step before clustering to potentially improve cluster quality or reduce computational cost, especially for high-dimensional data.

In dimensionality reduction, the goal is to find a lower dimensional representation of the data. There are generally a few reasons why one might want to do this:

1. **Computational efficiency.** Many algorithms are much faster on lower dimensional data.
2. **Improved learning.** Many algorithms' assumptions are better satisfied in low dimensions. Furthermore, dimensionality reduction can have a fairly strong "regularizing" effect. Think of something like Occam's razor as loose motivation here.
3. **Pretraining.** Learning compressed representations of unsupervised data is one of the core mechanisms by which pretraining can be accomplished. Pretraining is the process by which good features for supervised learning can be learned from very large amounts of unsupervised data. This is important because unsupervised data is often much easier to come by. For example, it's much easier to find pictures of houses than to find out what those specific houses were sold for.
4. **Visualization.** Plotting high dimensional data is hard. Plotting 2D or 3D data is easy. If you can reduce the dimensionality of your data down to 2 or 3 dimensions while preserving its structure, this can be useful for visualizing it.

Formally, we assume that  $\mathbf{x}_i \in \mathbb{R}^D$ , where  $D$  is a very high dimensional space. We'd like to find a compressed representation  $f(\mathbf{x}_i) = \mathbf{z}_i \in \mathbb{R}^d$  with  $d \ll D$ . Now, obviously this is not specified well enough: just map every  $\mathbf{x}_i$  to 0 and you've achieved this. The particular goal is that we'd like each  $\mathbf{z}_i$  to retain some of the "structure" in each  $\mathbf{x}_i$ . One common way to achieve this is to consider a second function  $g : \mathbb{R}^d \rightarrow \mathbb{R}^D$  that "reconstructs" the original input  $\mathbf{x}_i$ :

$$\underbrace{\mathbf{x}_i \rightarrow f(\mathbf{x}_i) \rightarrow \mathbf{z}_i \rightarrow g(\mathbf{z}_i) \rightarrow \hat{\mathbf{x}}_i}_{\text{Minimize } \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|}$$

We'll focus on **principal component analysis** (PCA) as the main dimensionality reduction method, and touch upon **autoencoders** in the generative modeling section.

## 1 Principal Component Analysis

To build intuition, consider a simple scenario: suppose your data points  $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^D$  roughly lie along a single direction  $\mathbf{u}$  in  $D$ -dimensional space, so that  $\mathbf{x}_i \approx \alpha_i \mathbf{u}$  for some scalar  $\alpha_i$ . Even

though each  $\mathbf{x}_i$  has  $D$  coordinates, you could describe each point well with just one number — how far along  $\mathbf{u}$  it sits. If you knew  $\mathbf{u}$ , you could compress the entire dataset from  $nD$  numbers down to  $n$  numbers (the  $\alpha_i$ 's) with little loss of information. PCA automates exactly this idea: it finds the directions along which the data varies the most, so that projecting onto those directions preserves as much information as possible.

More formally, suppose we are given  $n$  data points  $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^D$ , and  $d < D$  as our “target” dimensionality. Our goal is to find a projection  $f : \mathbb{R}^D \rightarrow \mathbb{R}^d$  that maps  $D$  dimensional inputs  $\mathbf{x}$  to  $d$  dimensional outputs  $\mathbf{z}$  while preserving structure of the data. In PCA, we aim to find a **linear projection** of the form:

$$\mathbf{z} := f(\mathbf{x}) = Q\mathbf{x}$$

where  $Q \in \mathbb{R}^{d \times D}$  (so  $Q$  has  $d$  rows and  $D$  columns; multiplying  $Q$  by a  $D$ -dimensional vector  $\mathbf{x}$  produces a  $d$ -dimensional vector  $\mathbf{z}$ ). The reconstructing function  $g(\cdot)$  uses the transpose  $Q^\top \in \mathbb{R}^{D \times d}$ , mapping back from  $d$  dimensions to  $D$ :

$$\hat{\mathbf{x}} := g(\mathbf{z}) = Q^\top \mathbf{z}$$

We further require the rows of  $Q$  to be **orthonormal** (unit length and mutually perpendicular). Why? If two rows pointed in similar directions, projecting onto both would capture nearly the same information — wasting one of our  $d$  dimensions. Orthogonality ensures each direction captures a genuinely new aspect of the data’s variation. Unit length is a normalization convention that keeps the math clean (otherwise the scale of each row would be a free parameter).

We’re going to introduce two alternative (and equivalent) views of PCA:

1. **Maximum variance:** PCA finds the  $d$  directions along which the projected data has the largest spread (variance).
2. **Best reconstruction:** If we are only allowed  $d$  dimensions to represent our  $D$ -dimensional data, PCA finds the  $d$  directions that let us reconstruct the original data as accurately as possible, i.e., minimizing  $\|Q^\top Q\mathbf{x} - \mathbf{x}\|_2^2$ .

Before developing either view, we need one prerequisite: PCA requires the data to be zero mean. In particular:

$$\bar{\mathbf{x}} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i = \mathbf{0}$$

To achieve this if  $\bar{\mathbf{x}} \neq \mathbf{0}$ , simply subtract the mean from each input  $\mathbf{x}_i$ :

$$\mathbf{x}_i \leftarrow \mathbf{x}_i - \bar{\mathbf{x}}$$

Note that this doesn’t really impact dimensionality reduction at all. Suppose  $g : \mathbb{R}^d \rightarrow \mathbb{R}^D$  is our function that reconstructs the original data from the low dimensional representation. Then simply taking  $g^+(\mathbf{z}) = g(\mathbf{z}) + \bar{\mathbf{x}}$  as the “real” reconstruction function adds the subtracted mean back.

Centering the data is crucial because PCA finds directions of maximum variance *around the data’s mean*. If the data is not centered, the first principal component might simply point from the origin towards the overall mean of the data cloud, rather than capturing the variance *within* the data cloud itself.

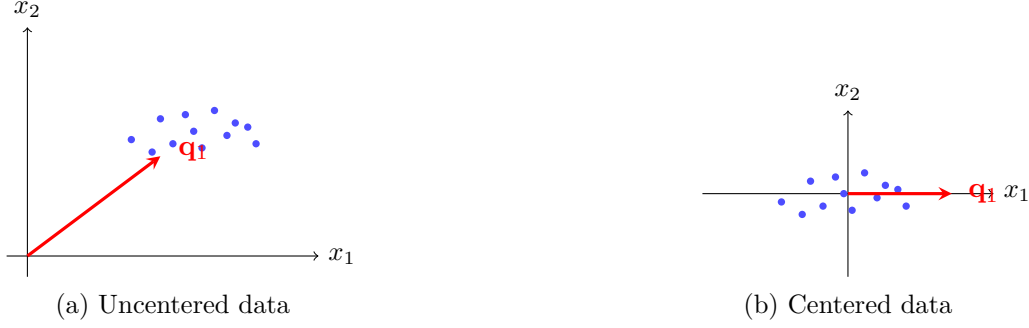


Figure 1: Effect of centering on PCA. (a) Without centering,  $\mathbf{q}_1$  points from the origin toward the data's mean rather than along its spread. (b) After centering,  $\mathbf{q}_1$  correctly captures the direction of maximum variance within the data.

### 1.1 PCA as Finding Maximum Variance Directions

To find directions that capture the data's 'spread' or structure, we need to understand how the features vary and co-vary. This information is captured by the sample covariance matrix  $S$ . As usual, denote by  $\mathbf{X}$  the  $n \times D$  matrix whose rows are the given examples  $\mathbf{x}_1, \dots, \mathbf{x}_n$  (after centering). The  $D \times D$  sample covariance matrix  $S$  can be written as:

$$S = \frac{1}{n-1} \mathbf{X}^\top \mathbf{X}$$

#### Note on the $1/(n-1)$ Factor (Bessel's Correction)

The factor  $1/(n-1)$  instead of  $1/n$  is Bessel's correction. It makes  $S$  an *unbiased estimator* of the true population covariance  $\Sigma$ , meaning  $\mathbb{E}[S] = \Sigma$ . This is necessary because we use the sample mean  $\bar{\mathbf{x}}$  (calculated from the data) instead of the true population mean  $\boldsymbol{\mu}$  when estimating covariance. Using  $1/n$  would result in a biased estimator that slightly underestimates the true variance (specifically,  $\mathbb{E}[\frac{1}{n} \sum_i (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^\top] = \frac{n-1}{n} \Sigma$ ). While this scaling doesn't affect the eigenvectors needed for PCA, using the unbiased estimator is standard statistical practice.

Now, imagine you want to project the data onto a direction defined by a unit vector  $\mathbf{q}_1$ . The **projection** of  $\mathbf{x}_i$  onto  $\mathbf{q}_1$  is given by:

$$(\mathbf{x}_i^\top \mathbf{q}_1) \mathbf{q}_1$$

Indeed, this is one geometric way to interpret the dot product: the dot product  $(\mathbf{x}_i^\top \mathbf{q}_1)$  measures the "shadow" that  $\mathbf{x}_i$  casts on  $\mathbf{q}_1$  assuming  $\mathbf{q}_1$  is a unit vector. Suppose we project each data point onto this vector  $\mathbf{q}_1$ . In other words, we construct the dataset:

$$[\mathbf{z}_1, \dots, \mathbf{z}_n] = [\mathbf{x}_1^\top \mathbf{q}_1, \dots, \mathbf{x}_n^\top \mathbf{q}_1]$$

First of all, before we talk about finding  $\mathbf{q}_1$ , note that this satisfies our definition of dimensionality reduction function  $f$  and reconstruction function  $g$  with  $Q = \mathbf{q}_1^\top$ :

$$\begin{aligned} \mathbf{z}_i &= f(\mathbf{x}_i) = \mathbf{q}_1^\top \mathbf{x}_i \\ \hat{\mathbf{x}}_i &= g(f(\mathbf{x}_i)) = Q^\top Q \mathbf{x}_i = \mathbf{q}_1 (\mathbf{q}_1^\top \mathbf{x}_i) = (\mathbf{x}_i^\top \mathbf{q}_1) \mathbf{q}_1 \end{aligned}$$

Here,  $f(\mathbf{x}_i)$  reduces  $\mathbf{x}_i$  to just how far along the single direction  $\mathbf{q}_1$  it "casts a shadow". The reconstruction gives exactly that vector back.

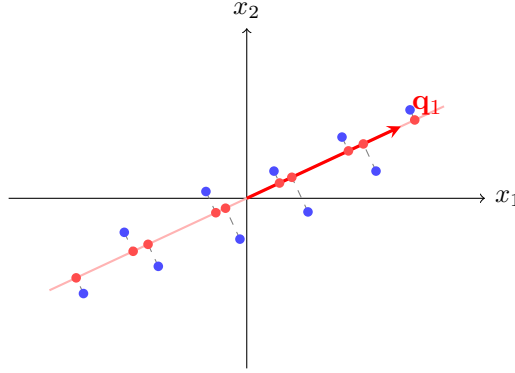


Figure 2: Projecting data onto the direction  $\mathbf{q}_1$ . Each blue point  $\mathbf{x}_i$  maps to its "shadow" (red) on  $\mathbf{q}_1$  via the scalar projection  $\mathbf{x}_i^\top \mathbf{q}_1$ . The dashed lines show the reconstruction error  $\|\hat{\mathbf{x}}_i - \mathbf{x}_i\|$ .

Now, what is the "best" vector  $\mathbf{q}_1$ ? One idea is to choose the vector that maximizes the **variance** of the values  $\mathbf{z}_1, \dots, \mathbf{z}_n$ . Intuitively, this is because the opposite would be really bad. If the values  $\mathbf{z}_1, \dots, \mathbf{z}_n$  have very small variance, that means that all of the original data  $\mathbf{x}_1, \dots, \mathbf{x}_n$  got mapped onto nearly the same value. In contrast, having high variance means that it's relatively "easy" to distinguish the data points  $\mathbf{x}_1, \dots, \mathbf{x}_n$  just from their projection onto  $\mathbf{q}_1$ .

The sample variance of  $\mathbf{z}_1, \dots, \mathbf{z}_n$  is given by (recall the data is zero mean, so  $\bar{\mathbf{z}} = 0$ ):

$$\begin{aligned} \frac{1}{n-1} \sum_{i=1}^n \mathbf{z}_i^2 &= \frac{1}{n-1} \sum_{i=1}^n (\mathbf{q}_1^\top \mathbf{x}_i)^2 && \text{(substituting } \mathbf{z}_i = \mathbf{q}_1^\top \mathbf{x}_i) \\ &= \frac{1}{n-1} \|\mathbf{X}\mathbf{q}_1\|_2^2 && \text{(the } i\text{th entry of } \mathbf{X}\mathbf{q}_1 \text{ is } \mathbf{x}_i^\top \mathbf{q}_1) \\ &= \frac{1}{n-1} \mathbf{q}_1^\top \mathbf{X}^\top \mathbf{X} \mathbf{q}_1 && \text{(since } \|\mathbf{v}\|_2^2 = \mathbf{v}^\top \mathbf{v}) \\ &= \mathbf{q}_1^\top \mathbf{S} \mathbf{q}_1 && \text{(since } \mathbf{S} = \frac{1}{n-1} \mathbf{X}^\top \mathbf{X}) \end{aligned}$$

The direction of **maximum** variance is given by the largest eigenvector of  $\mathbf{S}$  (recall:  $\mathbf{v}$  is an **eigenvector** of  $\mathbf{S}$  with eigenvalue  $\lambda$  if  $\mathbf{S}\mathbf{v} = \lambda\mathbf{v}$ , i.e.,  $\mathbf{S}$  simply scales  $\mathbf{v}$  by  $\lambda$ ). If you'd like to see a derivation of this fact, note that finding the direction of maximum variance involves solving:

$$\mathbf{q}_1 = \arg \max_{\mathbf{q}_1: \|\mathbf{q}_1\|_2^2=1} \mathbf{q}_1^\top \mathbf{S} \mathbf{q}_1$$

Introducing a Lagrange multiplier  $\lambda$  for the constraint  $\|\mathbf{q}_1\|_2^2 = 1$ , we obtain the Lagrangian function

$$-\mathbf{q}_1^\top \mathbf{S} \mathbf{q}_1 + \lambda(\mathbf{q}_1^\top \mathbf{q}_1 - 1)$$

Minimizing this over  $\mathbf{q}_1$  by taking the derivative and setting equal to zero gives:

$$\mathbf{S} \mathbf{q}_1 = \lambda \mathbf{q}_1,$$

indeed requiring that  $\mathbf{q}_1$  is an eigenvector of  $S$ . Moreover, the sample variance of  $\mathbf{z}_1, \dots, \mathbf{z}_n$  is then given by:

$$\mathbf{q}_1^\top S \mathbf{q}_1 = \lambda \mathbf{q}_1^\top \mathbf{q}_1 = \lambda,$$

i.e., the eigenvalue corresponding to this eigenvector. Geometrically, the eigenvectors of the covariance matrix point in the directions where the data varies the most, and the corresponding eigenvalues measure the amount of variance along those directions. The vector  $\mathbf{q}_1$  corresponding to the largest eigenvalue is called the first **principal component** (PC). To find  $d$  directions that maximize total variance, it turns out that indeed the correct strategy is to just take the top  $d$  eigenvectors of the covariance matrix  $S$ .

A naive implementation of PCA can therefore be described as follows:

1. First, zero mean the data and compute the sample covariance matrix  $S = \frac{1}{n-1} \mathbf{X}^\top \mathbf{X}$ .
2. Compute an eigendecomposition  $S = V \Lambda V^\top$ . Here,  $V$  is a  $D \times D$  orthogonal matrix whose columns are the eigenvectors of  $S$ , and  $\Lambda$  is a  $D \times D$  diagonal matrix containing the eigenvalues.
3. Take  $Q^\top$  to be the first  $d$  columns of  $V$ , e.g. the first  $d$  eigenvectors.
4. Define  $\mathbf{z} := f(\mathbf{x}) = Q\mathbf{x}$  and  $\hat{\mathbf{x}} := g(\mathbf{z}) = Q^\top \mathbf{z}$ .

Here, the low dimensional representation  $\mathbf{z}_i$  of the input  $\mathbf{x}_i$  is given by the inner products of  $\mathbf{x}_i$  with each of the eigenvectors:

$$\mathbf{z}_i = [\mathbf{x}_i^\top \mathbf{q}_1, \dots, \mathbf{x}_i^\top \mathbf{q}_d]$$

Since there are  $d$  eigenvectors that we kept, and each inner product is a single real value,  $\mathbf{z}_i$  ends up being  $d$  dimensional.

**A better implementation.** The above implementation has two bottlenecks: (1) forming  $S = \frac{1}{n-1} \mathbf{X}^\top \mathbf{X}$  costs  $O(nD^2)$  time, and (2) eigendecomposing the full  $D \times D$  matrix  $S$  costs  $O(D^3)$  time — even though we only need the top  $d$  eigenvectors, not all  $D$  of them.

The **singular value decomposition** (SVD) sidesteps both issues. Every matrix  $\mathbf{X}$  can be factored as  $\mathbf{X} = U \Sigma V^\top$ , where  $U$  ( $n \times n$ ) and  $V$  ( $D \times D$ ) are orthogonal matrices and  $\Sigma$  ( $n \times D$ ) is diagonal with nonnegative entries  $\sigma_1 \geq \sigma_2 \geq \dots$  called the **singular values**. The columns of  $V$  turn out to be exactly the eigenvectors of  $S$ , because:

$$\mathbf{X}^\top \mathbf{X} = V \Sigma^\top \Sigma V^\top$$

which is the eigendecomposition of  $\mathbf{X}^\top \mathbf{X}$  with eigenvalues  $\sigma_j^2$  (so  $\lambda_j = \sigma_j^2 / (n-1)$ ). The key practical advantage is that we can compute a **truncated SVD** that finds only the top  $d$  singular vectors of  $\mathbf{X}$  directly, without ever forming  $S$ , reducing the cost to  $O(nDd)$  — a big win when  $d \ll D$ .<sup>1</sup>

---

<sup>1</sup>The core idea behind truncated SVD is *power iteration*: repeatedly multiplying a random vector by  $\mathbf{X}^\top \mathbf{X}$  causes it to align with the top eigenvector, because the largest eigenvalue dominates. Each multiply costs  $O(nD)$  (two matrix-vector products with  $\mathbf{X}$  and  $\mathbf{X}^\top$ ), and we never form the  $D \times D$  matrix  $S$ . Repeating for  $d$  vectors (via deflation or a block variant) gives  $O(nDd)$  overall.

## 1.2 PCA as Minimizing Reconstruction Error

An alternative — and equivalent — view of PCA is based on minimizing the reconstruction error. Let's see why for a single direction  $\mathbf{q}$  (the  $d = 1$  case). The reconstruction of  $\mathbf{x}_i$  onto a unit vector  $\mathbf{q}$  is  $\hat{\mathbf{x}}_i = (\mathbf{x}_i^\top \mathbf{q})\mathbf{q}$ . The reconstruction error is:

$$\begin{aligned}\|\hat{\mathbf{x}}_i - \mathbf{x}_i\|_2^2 &= \|(\mathbf{x}_i^\top \mathbf{q})\mathbf{q} - \mathbf{x}_i\|_2^2 \\ &= (\mathbf{x}_i^\top \mathbf{q})^2 \underbrace{\|\mathbf{q}\|_2^2}_{=1} - 2(\mathbf{x}_i^\top \mathbf{q})(\mathbf{q}^\top \mathbf{x}_i) + \|\mathbf{x}_i\|_2^2 \\ &= \|\mathbf{x}_i\|_2^2 - (\mathbf{x}_i^\top \mathbf{q})^2\end{aligned}$$

Since  $\|\mathbf{x}_i\|_2^2$  doesn't depend on  $\mathbf{q}$ , the total reconstruction error  $\sum_{i=1}^n \|\hat{\mathbf{x}}_i - \mathbf{x}_i\|_2^2 = \sum_i \|\mathbf{x}_i\|_2^2 - \sum_i (\mathbf{x}_i^\top \mathbf{q})^2$  is minimized by maximizing  $\sum_i (\mathbf{x}_i^\top \mathbf{q})^2$  — which, up to the factor  $\frac{1}{n-1}$ , is exactly the projected variance  $\mathbf{q}^\top S \mathbf{q}$  we maximized in the previous section! So the first eigenvector of  $S$  simultaneously maximizes projected variance and minimizes reconstruction error.

This generalizes to  $d$  directions (you will prove the general case in HW3): for  $Q \in \mathbb{R}^{d \times D}$  with orthonormal rows, the same identity holds with  $(\mathbf{x}_i^\top \mathbf{q})^2$  replaced by  $\|Q\mathbf{x}_i\|_2^2$ , and the top- $d$  eigenvectors of  $S$  remain optimal. The quality of a  $d$ -dimensional PCA approximation can be summarized by the **fraction of variance explained**:

$$\frac{\lambda_1 + \dots + \lambda_d}{\lambda_1 + \dots + \lambda_D}$$

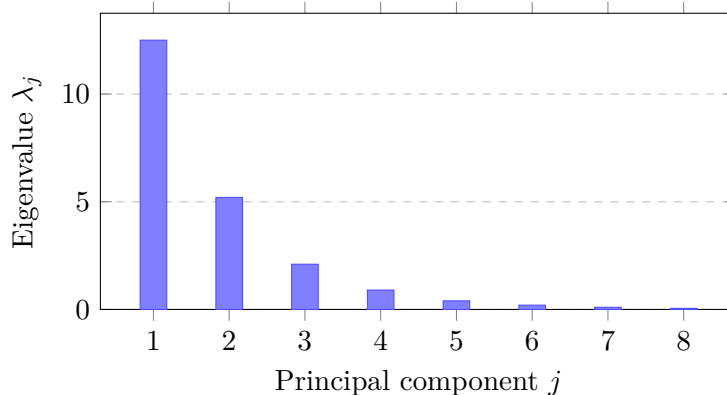


Figure 3: Eigenvalues of  $S$  in decreasing order. The sharp drop-off after the first few components suggests that most of the data's variance is captured by a small number of principal components.

**Choosing  $d$  in practice.** A common heuristic is to plot the eigenvalues  $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_D$  in decreasing order (Figure 3) and look for a point where they drop off sharply. Alternatively, one chooses  $d$  so that the fractional reconstruction accuracy exceeds a desired threshold (e.g., 0.95), meaning 95% of the total variance is retained.

**Limitations of PCA.** PCA finds only *linear* projections, so it cannot capture nonlinear structure in the data. It is also sensitive to the scaling of features: if one feature has a much larger range

than the others, PCA will disproportionately weight that feature. For this reason, it is common to *standardize* each feature (subtract its mean and divide by its standard deviation) rather than merely centering. Autoencoders, which we will cover next, address the linearity limitation by using neural networks as the encoder  $f$  and decoder  $g$ .